

Simulacija naelektrenih delcev

Priprava na zagovor projekta – Programiranje 3

Gregor Antonaz – 89201088

April 2026

Kazalo

1	Pregled problema	3
1.1	Coulombova sila (modificirana)	3
1.2	Mejna sila	3
1.3	Posodobitev stanja (Eulerjev integracija)	3
2	Struktura projekta	4
2.1	Ključni razredi	4
2.1.1	Particle.java	4
2.1.2	Simulation.java (vmesnik)	5
2.1.3	AbstractSimulation.java	5
2.1.4	ForceKernel.java	6
3	Sekvenčna implementacija	6
3.1	Algoritem	6
3.2	Kompleksnost	6
4	Vzporedna implementacija (Parallel)	7
4.1	Strategija	7
4.2	Razdelitev dela	7
4.3	Sinhronizacija	7
4.4	Samodejno prilagajanje strojni opremi	8
4.5	Pohitritev	8
5	Porazdeljena implementacija (MPJ Express)	9
5.1	Kaj je MPJ Express?	9
5.2	Zakaj ločen DistributedRunner?	9
5.3	Podatkovne strukture za MPI	9
5.4	Razdelitev delcev med procese	10
5.5	Algoritem enega cikla	10
5.6	Allgatherv – ključna MPI operacija	11
5.7	Overhead porazdeljene verzije	12
6	Primerjava implementacij	12
6.1	Meritve	12

7	Načrtovalski vzorci	13
8	Zagon programa	14
8.1	Zagonske skripte	14
8.2	Argumenti	14
8.3	Preverjanje pravilnosti	14
8.4	Porazdeljeni način iz grafičnega vmesnika	14
9	Pogosta vprašanja za zagovor	15

1 Pregled problema

Simuliramo sistem n naelektrenih delcev v 2D ravnini znotraj omejenega pravokotnika. Vsak delec ima:

- Pozicijo (x, y)
- Hitrost (v_x, v_y)
- Naboj c (pozitiven ali negativen)
- Maso $m = 1.0$

1.1 Coulombova sila (modificirana)

Sila med delcema i in j je:

$$F = \frac{|c_i \cdot c_j|}{d^2}$$

kjer je d evklidska razdalja med njima: $d = \sqrt{(x_j - x_i)^2 + (y_j - y_i)^2}$.

Ključni koncept

POZOR – “spin” problema: V naši simulaciji velja **obratna logika** kot v pravi fiziki:

- Delca z **enakim** predznakom naboja se **privlačita**
- Delca z **različnim** predznakom naboja se **odbijata**

To je nasprotno od Coulombovega zakona v fiziki, vendar je tako definirano v nalogi (poglavje 11.2.1, Andrews).

1.2 Mejna sila

Pravokotnik ima naboj, ki odbija delce. Ko se delec približa robu (manj kot 5 enot), deluje nanj odbojna sila:

$$F_{\text{meja}} = \frac{10.0}{\text{dist}^2}$$

To prepreči, da bi delci zapustili simulacijski prostor.

1.3 Posodobitev stanja (Eulerjev integracija)

Vsak cikel:

1. Izračunamo vse sile na vsak delec
2. Pospešek: $a = F/m$
3. Nova hitrost: $v' = v + a$
4. Nova pozicija: $x' = x + v'$

Možno vprašanje na zagovoru

Zakaj se najprej izračunajo **vse** sile, šele nato posodobijo pozicije?

Odgovor: Če bi pozicije posodabljali sproti (med izračunom sil), bi nekateri delci že imeli nove pozicije, drugi pa še stare. Rezultat bi bil nedosleden – odvisen od vrstnega reda obdelave. Z dvofaznim pristopom zagotovimo, da vsi delci “vidijo” enako stanje v istem ciklu.

2 Struktura projekta

```
src/main/java/com/example/chargedparticles/
  SimulationRunner.java      # vstopna točka (seq + parallel)
                             # + skupna simulacijska zanka runCycles()
  DistributedRunner.java     # vstopna točka za MPJ
model/
  Particle.java             # podatkovni model delca
simulation/
  ForceKernel.java          # izracun sil in integracija (skupno jedro)
  Simulation.java           # vmesnik
  AbstractSimulation.java   # skupno stanje vseh treh načinov
  SequentialSimulation.java # sekvencna impl.
  ParallelSimulation.java   # vzporedna impl.
  DistributedSimulation.java # porazdeljena impl. (MPJ)
  SimulationFactory.java    # factory vzorec
  SimulationMode.java       # enum: SEQUENTIAL, PARALLEL, DISTRIBUTED
  SimulationParameters.java # parametri + branje ukazne vrstice
ui/
  SimulationUI.java         # Swing GUI
```

Ključni koncept

Vsi trije načini uporabljajo **isto računsko jedro** (ForceKernel) nad **isto podatkovno postavitvijo** in **isto simulacijsko zanko** (SimulationRunner.runCycles). Razlikujejo se samo v tem, **kdo** izračuna **kateri odsek** delcev. Zato izmerjene razlike v času izvirajo iz načina delitve dela in ne iz razlik v izvedbi fizike.

2.1 Ključni razredi

2.1.1 Particle.java

Preprost POJO z lastnostmi `x`, `y`, `vx`, `vy`, `charge`, `mass`. Ima statično tovarniško metodo `randomParticle()` ki sprejme `Random` objekt za deterministično generiranje.

Možno vprašanje na zagovoru

Zakaj uporabljamo `Random` z seed-om 42?

Odgovor: Da zagotovimo **reproducibilnost**. Vsi trije načini (sekvenčni, vzporedni, porazdeljeni) začnejo z **enako** začetno razporeditvijo delcev, kar omogoča pravilno primerjavo rezultatov in časov izvajanja. Če seed ne bi bil enak, ne bi mogli potrditi, da implementacije dajejo enake rezultate.

2.1.2 Simulation.java (vmesnik)

Definira pogodbo za vse implementacije:

```
1 public interface Simulation {
2     void performOneCycle();
3     void syncToParticles(List<Particle> particles);
4     String getDescription();
5     default void shutdown() { }
6 }
```

`performOneCycle()` izvede en cikel nad internim stanjem. `syncToParticles()` to stanje prepiše v seznam `Particle` objektov – klicemo ga samo pred izrisom ali izpisom, ne v vroči zanki.

Možno vprašanje na zagovoru

Zakaj `performOneCycle()` nima parametrov?

Odgovor: Delci in parametri se ne spreminjajo med tekom, zato jih simulacija prejme v **konstruktorju** in shrani. Tako v vsakem ciklu ni ponovnega prenosa argumentov niti preverjanja, ali je stanje že inicializirano.

Možno vprašanje na zagovoru

Zakaj smo uporabili vmesnik?

Odgovor: Vmesnik omogoča **polimorfizem** – `SimulationRunner` dela z `Simulation` referenco in ne ve, katera implementacija teče. To omogoča enostavno preklapljanje med načini brez spreminjanja klicne kode. Gre za **Strategy** načrtovalski vzorec.

2.1.3 AbstractSimulation.java

Skupna osnova vseh treh načinov. Hrani stanje, ki je pri vseh enako:

```
1 protected final int n; // stevilo delcev
2 protected final double[] state; // 6 doublov na delec
3 protected final double[] forces; // forces[i*2], forces[i*2+1]
4 protected final SimulationParameters params;
```

Ker je stanje skupno, se podrazredi razlikujejo samo po tem, kako razdelijo delo. Tu je tudi `syncToParticles()`, ki je pri vseh treh načinih enak.

2.1.4 ForceKernel.java

Statični razred, ki vsebuje **vso fiziko** simulacije. Uporabljajo ga vsi trije načini, zato so enačbe zapisane na enem samem mestu.

Podatkovna postavitev. Delci niso shranjeni kot objekti, ampak v **ravnem polju** `double[]`, 6 vrednosti na delec:

```
1 // [x, y, vx, vy, charge, mass] -> 6 doubleov na delec
2 // delec i se začne na odmiku i * 6
```

Dve operaciji:

- `computeForces(state, n, start, end, forces, params)` – izračuna sile za delce v območju `[start, end)` proti **vsem** `n` delcem, vključno z mejnimi silami
- `integrate(state, start, end, forces)` – posodobi hitrosti in pozicije za isto območje

Ključni koncept

Polje `forces` je **globalno indeksirano**: rezultat za delec `i` gre vedno v `forces[i*2]` in `forces[i*2+1]`, ne glede na to, kateri izvajalec ga je izračunal. Ker so območja različnih izvajalcev disjunktna, lahko več niti hkrati piše v isto polje **brez zaklepanja**.

Logika predznaka pri Coulombovi sili:

```
1 double sign = (c1 * c2 >= 0) ? 1.0 : -1.0;
2 fxSum += sign * magnitude * (dx / r);
3 fySum += sign * magnitude * (dy / r);
```

Ključni koncept

`dx = p2.x - p1.x`. Če je `sign = 1.0` (enak naboj), je sila v smeri **proti** `p2` = **privlačenje**. Če je `sign = -1.0` (različen naboj), je sila **stran** od `p2` = **odbijanje**.

3 Sekvenčna implementacija

`SequentialSimulation` – najenostavnejša možna implementacija.

3.1 Algoritem

1. **Faza 1 – Izračun sil:** Za vsak delec `i` izračunamo skupno silo od vseh ostalih delcev `j` in mejne sile. Shranimo v polje `forces[i][2]`.
2. **Faza 2 – Posodobitev:** Za vsak delec posodobimo hitrost in pozicijo.

3.2 Kompleksnost

- Faza 1: $O(n^2)$ – vsak delec proti vsem ostalim
- Faza 2: $O(n)$ – linearen prehod

- Skupaj na cikel: $O(n^2)$
- Skupaj za simulacijo: $O(n^2 \cdot C)$ kjer je C število ciklov

Celotna sekvenčna implementacija je zato samo dva klica jedra nad celotnim območjem $[0, n)$:

```
1 public void performOneCycle() {
2     ForceKernel.computeForces(state, n, 0, n, forces, params);
3     ForceKernel.integrate(state, 0, n, forces);
4 }
```

Možno vprašanje na zagovoru

Zakaj je sekvenčna različica tako kratka?

Odgovor: Ker je vsa fizika v `ForceKernel`, sekvenčna različica pa samo določi, da eno samo območje $[0, n)$ obdelata ena nit. Vzporedna in porazdeljena različica kličeta **isti dve metodi**, le z drugimi mejami območja. To je tudi razlog, da vse tri dajo popolnoma enak rezultat.

4 Vzporedna implementacija (Parallel)

`ParallelSimulation` – uporaba `ExecutorService` za paralelizacijo izračuna sil.

4.1 Strategija

Ključni koncept

Paraleliziramo **Fazo 1** (izračun sil). Faza 2 (posodobitev pozicij) ostane sekvenčna – je $O(n)$ in zanemarljivo hitra v primerjavi s Fazo 1 ki je $O(n^2)$.

4.2 Razdelitev dela

Delce razdelimo v **kose** (chunks):

```
1 int chunkSize = (n + numThreads - 1) / numThreads;
2 // nit 0: delci [0, chunkSize)
3 // nit 1: delci [chunkSize, 2*chunkSize)
4 // ...
```

Vsaka nit izračuna sile za **svoj kos** delcev, a pri tem prebere **vse** delce (ker vsak delec vpliva na vsakega drugega).

4.3 Sinhronizacija

Naloge (po ena na kos) zgradimo **enkrat** v konstruktorju, saj se ne spreminjajo. V vsakem ciklu jih samo oddamo:

```
1 public void performOneCycle() {
2     executor.invokeAll(tasks);    // pregrada: caka na vse niti
3     ForceKernel.integrate(state, 0, n, forces);
4 }
```

`invokeAll()` odda vse naloge in se vrne **šele ko so vse končane** – deluje torej kot pregrada (barrier) med Fazo 1 in Fazo 2.

Možno vprašanje na zagovoru

Zakaj `invokeAll()` in ne `submit()` z `CountDownLatch`?

Odgovor: `invokeAll()` že vsebuje pregrado, zato števca ni treba ročno ustvarjati, zmanjševati in čakati nanj. Manj kode pomeni manj možnosti za napako – npr. pozabljen `countDown()` v veji z izjemo bi glavno nit blokiral za vedno.

Možno vprašanje na zagovoru

Zakaj ni **race condition** pri pisanju v `forces[i]`?

Odgovor: Vsaka nit piše v **svoj del** polja `forces` – nit 0 piše v indekse $[0, \text{chunkSize})$, nit 1 v $[\text{chunkSize}, 2 \cdot \text{chunkSize})$ itd. Ker se indeksi ne prekrivajo, ni potrebe po zaklepanju.

Možno vprašanje na zagovoru

Zakaj Fazo 2 izvajamo sekvenčno?

Odgovor: Faza 2 je $O(n)$ – linearna. Pri $n = 3000$ je to 3000 operacij. Faza 1 je $O(n^2) = 9.000.000$ operacij. Paralelizacija Faze 2 bi prinesla zanemarljiv prihranek, hkrati pa bi uvedla kompleksnost sinhronizacije.

4.4 Samodejno prilagajanje strojni opremi

```
1 this.numThreads = Runtime.getRuntime().availableProcessors();
2 this.executor = Executors.newFixedThreadPool(numThreads);
```

`availableProcessors()` vrne število logičnih jeder. Na mojem računalniku je to 10.

Možno vprašanje na zagovoru

Zakaj `FixedThreadPool` in ne `CachedThreadPool`?

Odgovor: `FixedThreadPool` ohranja konstantno število niti skozi celotno simulacijo. Ne ustvarja novih niti za vsak cikel – niti se ponovno uporabijo. `CachedThreadPool` bi lahko ustvaril preveč niti in povzročil overhead.

4.5 Pohitritev

Idealna pohitritev z T nitmi je $T\times$. V praksi je manjša zaradi:

- **Overhead** – oddajanje nalog in pregrada `invokeAll` v vsakem ciklu
- **Neenakomerna razdelitev** – če n ni deljiv s številom niti
- **Amdahlov zakon** – Faza 2 je sekvenčna in omejuje maksimalno pohitritev

Ključni koncept

Amdahlov zakon: Če je delež sekvenčne kode s , je maksimalna pohitritev:

$$S = \frac{1}{s + \frac{1-s}{p}}$$

V našem primeru je $s \approx \frac{O(n)}{O(n^2)} \approx \frac{1}{n}$, kar je zelo majhno – zato se vzporedna verzija dobro skalira.

5 Porazdeljena implementacija (MPJ Express)

DistributedSimulation + DistributedRunner – uporaba MPI (Message Passing Interface) preko MPJ Express.

5.1 Kaj je MPJ Express?

MPJ Express je Java implementacija MPI standarda. Omogoča komunikacijo med več procesi, ki tečejo na istem ali različnih računalnikih.

Možno vprašanje na zagovoru

Kaj je razlika med vzporedno in porazdeljeno implementacijo?

Odgovor:

- **Vzporedna:** Več **niti** v enem procesu, deljeni pomnilnik. Niti direktno berejo isti `List<Particle>`.
- **Porazdeljena:** Več ločenih **procesov**, vsak s svojim pomnilnikom. Procesi morajo **eksplicitno komunicirati** (MPI) za izmenjavo podatkov.

5.2 Zakaj ločen DistributedRunner?

MPJ Express zahteva `MPI.Init(args)` na samem začetku programa. Vsi procesi zazenejo isti **program** (SPMD – Single Program, Multiple Data). Zato imamo ločen vstopni razred, ki se zažene z:

```
mpjrun.sh -np 4 com.example.chargedparticles.DistributedRunner
```

5.3 Podatkovne strukture za MPI

MPI ne more pošiljati Java objektov (`Particle`). Zato pretvorimo vse delce v **ravno polje** `double[]`:

```
1 // 6 vrednosti na delec: x, y, vx, vy, charge, mass
2 double[] state = new double[n * 6];
3
4 // Delec i je na odmiku i * 6:
5 // state[i*6 + 0] = x
6 // state[i*6 + 1] = y
```

```

7 // state[i*6 + 2] = vx
8 // state[i*6 + 3] = vy
9 // state[i*6 + 4] = charge
10 // state[i*6 + 5] = mass

```

Možno vprašanje na zagovoru

Zakaj ne serializiramo `Particle` objektov?

Odgovor: Serializacija Java objektov je počasna (overhead marshalling/unmarshalling). Ravno `double[]` polje je najhitrejši način za MPI komunikacijo – `MPI.DOUBLE` je primitiven tip, ki se kopira direktno brez pretvorb.

5.4 Razdelitev delcev med procese

Delce razdelimo čim bolj enakomerno:

```

1 // Zacetni indeks delcev za proces r.
2 // Prvih (n mod size) procesov dobi en delec vec.
3 private int startIndexOf(int r) {
4     int base = n / size;
5     int remainder = n % size;
6     return (r < remainder)
7         ? r * (base + 1)
8         : remainder * (base + 1) + (r - remainder) * base;
9 }
10
11 // Moje obmocje je preprosto:
12 myStart = startIndexOf(rank);
13 myEnd   = startIndexOf(rank + 1);

```

Primer z 10 delci in 3 procesi:

Rank	Delci	Število
0	[0, 4)	4
1	[4, 7)	3
2	[7, 10)	3

5.5 Algoritem enega cikla

1. **Faza 1:** Vsak proces izračuna sile za **svoje** delce. Za izračun potrebuje podatke **vseh** delcev (ki jih že ima v `state`).
2. **Faza 2:** Vsak proces posodobi pozicije in hitrosti **samo svojih** delcev v `state`.
3. **Faza 3 – Allgatherv:** Vsi procesi si izmenjajo posodobljene podatke.

5.6 Allgatherv – ključna MPI operacija

Ključni koncept

Allgatherv = “vsi zberejo vse, različne količine”.

Vsak proces pošlje svoj del podatkov in prejme dele od vseh ostalih. Po klicu ima vsak proces **celoten posodobljen state**.

```
1 MPI.COMM_WORLD.Allgatherv(
2     state, sendOffset, sendCount, MPI.DOUBLE, // posiljam
3     state, 0, recvCounts, displacements, MPI.DOUBLE // prejemam
4 );
```

Parametri:

- `sendOffset` – začetek mojih podatkov v polju
- `sendCount` – koliko doublov pošiljam
- `recvCounts[]` – koliko doublov pričakujem od vsakega procesa
- `displacements[]` – kje v sprejemnem polju se začnejo podatki vsakega procesa

Pred Allgatherv:

```
Rank 0: [x0 y0 ... | stari | stari ] <- posodobil svoj del
Rank 1: [stari | x4 y4 ... | stari ] <- posodobil svoj del
Rank 2: [stari | stari | x7 y7 ...] <- posodobil svoj del
```

Po Allgatherv:

```
Rank 0: [x0 y0 ... | x4 y4 ... | x7 y7 ...] <- vsi novi
Rank 1: [x0 y0 ... | x4 y4 ... | x7 y7 ...] <- vsi novi
Rank 2: [x0 y0 ... | x4 y4 ... | x7 y7 ...] <- vsi novi
```

Slika 1: Vizualizacija Allgatherv operacije

Možno vprašanje na zagovoru

Zakaj ne uporabimo Scatter + Gather namesto Allgatherv?

Odgovor: Pri Scatter/Gather bi samo rank 0 (master) imel celoten rezultat in bi ga moral ponovno razpošljati (Broadcast). Z Allgatherv **vsi** procesi hkrati prejmejo celotno posodobljeno stanje – ena operacija namesto dveh. To je učinkovitejše.

Možno vprašanje na zagovoru

Zakaj potrebuje vsak proces **vse** delce, ne le svoje?

Odgovor: Ker sila na delec i je vsota sil od **vseh** ostalih $n - 1$ delcev. Delec na rank 0 čuti silo od delcev na rank 1, rank 2, itd. Brez poznavanja pozicij vseh delcev izračun ni možen. To je inherentna lastnost N-body problema.

5.7 Overhead porazdeljene verzije

Porazdeljena verzija je počasnejša od vzporedne zaradi:

1. **Allgatherv vsak cikel** – kopiranje $6n$ doublov med vsemi procesi
2. **MPI overhead** – inicializacija, interna sinhronizacija
3. **Ločen pomnilnik** – ni možnosti deljenja podatkov brez komunikacije

Vendar ima porazdeljena verzija prednost: lahko teče na **več računalnikih**, kar omogoča skaliranje prek zmogljivosti enega stroja.

6 Primerjava implementacij

	Sekvenčna	Vzporedna	Porazdeljena
Enot izvajanja	1 nit	T niti	P procesov
Pomnilnik	deljen	deljen	ločen
Komunikacija	/	deljen pomnilnik	MPI Allgatherv
Sinhronizacija	/	invokeAll pregrada	Allgatherv
Vstopna točka	SimulationRunner	SimulationRunner	DistributedRunner
Zagon	java ...	java ...	mpjrun.sh -np P

6.1 Meritve

Meritve zaganjamo z `./benchmark.sh`. Vsaka nastavitev se izvede večkrat, poročamo najkrajši čas (druga dejavnost na računalniku lahko čas samo poveča, nikoli zmanjša).

2000 delcev, 1000 ciklov:

Način	Čas [s]	Pohitritev
Sekvenčna	6.94	1.0×
Porazdeljena (4 proc.)	2.56	2.7×
Porazdeljena (10 proc.)	2.39	2.9×
Vzporedna (10 niti)	1.45	4.8×

3000 delcev, 1000 ciklov:

Način	Čas [s]	Pohitritev
Sekvenčna	15.67	1.0×
Porazdeljena (4 proc.)	7.72	2.0×
Porazdeljena (10 proc.)	5.42	2.9×
Vzporedna (10 niti)	4.16	3.8×

Ključni koncept

Vzporedna je vedno najhitrejša na enem računalniku, in to je pričakovano: obe različici razdelita enako delo med enako število izvajalcev, a porazdeljena mora vsak cikel še prekopirati stanje med procesi.

Zanimivejši je **trend**. Zaostanek porazdeljene različice (10 proc.) za vzporedno pada z velikostjo problema:

n	Porazdeljena počasnejša za
500	278 %
1000	71 %
2000	64 %
3000	30 %

Razlog: komunikacija raste kot $O(n)$, računanje pa kot $O(n^2)$. Večji kot je problem, manjši delež časa gre za komunikacijo.

Možno vprašanje na zagovoru

Zakaj je pri $n = 500$ porazdeljena z 10 procesi počasnejša od **sekvenčne** (pohitritev $0,72\times$)?

Odgovor: Pri 500 delcih je računanja na cikel tako malo, da 1000 klicev `Allgather` prevlada nad celotnim časom izvajanja. Komunikacijski strošek je večji od prihranka pri računanju. To je najlepši primer v naših podatkih, da paralelizacija ni brezplačna.

Možno vprašanje na zagovoru

Zakaj je pri $n = 500$ bolje uporabiti 4 procese kot 10?

Odgovor: Manj procesov pomeni cenejšo kolektivno operacijo. Pri $n = 500$ so 4 procesi hitrejši (0,25 s proti 0,61 s), pri $n = 3000$ pa 10 procesov (5,42 s proti 7,72 s) – takrat je računanja dovolj, da se dodatni procesi izplačajo.

7 Načrtovalski vzorci

1. **Strategy** – vmesnik `Simulation` z različnimi implementacijami. `SimulationRunner` ne ve, katera implementacija teče.
2. **Factory** – `SimulationFactory.createSimulation(mode)` ustvari pravilno implementacijo glede na izbrani način.
3. **SPMD** (Single Program, Multiple Data) – pri porazdeljeni verziji vsi procesi zaze-nejo isti program (`DistributedRunner`), a obdelajo različne podatke glede na svoj rang.

8 Zagon programa

8.1 Zagonske skripte

Vsi trije načini se zaženejo z eno skripto na način. Skripte same poiščejo `JAVA_HOME` in `MPJ_HOME` (prek skupne `setup_env.sh`) in prevedejo projekt.

```
./run_sequential.sh          # sekvenčno, z graficnim vmesnikom
./run_parallel.sh           # vzporedno
./run_distributed.sh 4      # porazdeljeno, 4 MPI procesi

# Meritev brez izrisa
./run_distributed.sh 4 --particles 2000 --cycles 1000 --ui false

# Vse meritve iz porocila naenkrat
./benchmark.sh
```

8.2 Argumenti

Opcije so pri vseh treh skriptah **enake** – bere jih ista metoda `SimulationParameters.fromArgs()`.

Argument	Opis	Privzeto
-mode	sequential parallel	sequential
-particles N	število delcev	400
-cycles N	število ciklov	1000
-ui true false	graficni vmesnik	true
-window W H	velikost okna	800 600
-bounds x1 x2 y1 y2	meje prostora	0 800 0 600
-fps N	hitrost osveževanja	60

8.3 Preverjanje pravilnosti

Ker vsi načini startajo iz istega seeda, morajo dati **enak** rezultat:

```
./run_sequential.sh --particles 500 --cycles 300 --ui false
./run_parallel.sh --particles 500 --cycles 300 --ui false
./run_distributed.sh 4 --particles 500 --cycles 300 --ui false
```

Izpisanih pet delcev na koncu se mora v vseh treh primerih ujemati do zadnje decimalke.

8.4 Porazdeljeni način iz graficnega vmesnika

V oknu izberemo način **Porazdeljena (MPI)**; pojavi se polje **Procesi**. Klik na **Start** pže ukaz, ki bo izveden, in po potrditvi zazhene `run_distributed.sh` – ta odpre **novokno**, v katerem teče porazdeljena simulacija.

Možno vprašanje na zagovoru

Zakaj se porazdeljeni način ne izvede kar v istem oknu?

Odgovor: Ker potrebuje **več ločenih procesov**, ki jih mora zagnati `mpjrun.sh`. Obstoječi proces se ne more sam “razmnožiti” v MPI skupino – MPI okolje mora obstajati že ob zagonu programa (`MPI.Init()` je prvi klic v `DistributedRunner`).

9 Pogosta vprašanja za zagovor

Možno vprašanje na zagovoru

Kako zagotovite, da vse tri verzije dajo enake rezultate?

Odgovor: Vse tri verzije uporabljajo enak seed (42) za generiranje začetnih delcev, isto računsko jedro (`ForceKernel`) in enak dvofazni algoritem. Razlika je samo v tem, **kdo** izračuna katere sile – rezultat je enak, ker je vsota sil neodvisna od vrstnega reda seštevanja (do numerične natančnosti).

Možno vprašanje na zagovoru

Kaj je Amdahlov zakon in kako vpliva na vašo simulacijo?

Odgovor: Amdahlov zakon pravi, da je pohitritev omejena z deležem kode, ki **mora** teči sekvenčno. V našem primeru je Faza 2 (posodobitev, $O(n)$) sekvenčna, a je zanemarljivo majhna v primerjavi s Fazo 1 ($O(n^2)$), zato se dobro skaliramo.

Možno vprašanje na zagovoru

Zakaj je porazdeljena verzija počasnejša od vzporedne?

Odgovor: Vzporedna verzija deli pomnilnik – niti direktno berejo isti seznam delcev brez kopiranja. Porazdeljena verzija mora vsak cikel izvesti `Allgatherv`, ki kopira $6 \cdot n$ dublov med vsemi procesi. Ta komunikacijski overhead je še posebej izrazit pri lokalnem zagonu (multicore), kjer ni dejanske mrežne prednosti.

Možno vprašanje na zagovoru

Kdaj bi bila porazdeljena verzija hitrejša od vzporedne?

Odgovor: Ko imamo **več računalnikov** – na primer 4 računalnike z vsak 10 jedri. Vzporedna verzija je omejena na 10 niti enega računalnika, porazdeljena pa bi uporabila $4 \times 10 = 40$ jeder.

Možno vprašanje na zagovoru

Kako bi izboljšali časovno kompleksnost?

Odgovor: Z uporabo **prostorskega particioniranja** (Barnes-Hut algoritem s quadtree), ki zmanjša kompleksnost iz $O(n^2)$ na $O(n \log n)$. Delce, ki so daleč stran, obravnavamo kot eno skupino namesto posamezno.

Možno vprašanje na zagovoru

Zakaj GUI teče neodvisno od simulacije?

Odgovor: Izris poteka v ločenem timerju (Swing Timer na EDT – Event Dispatch Thread), simulacija pa v svoji niti. Tako GUI ne blokira izračuna in ostaja odziven. Če bi izračun tekkel na EDT, bi bil vmesnik zamrznjen.

Možno vprašanje na zagovoru

Zakaj se niti v `ParallelSimulation` ne “stepejo na prste” pri pisanju v `forces[]`?

Odgovor: Ker vsaka nit piše v **ekskluziven del** polja. Nit t piše samo v indekse $[t \cdot \text{chunkSize}, (t + 1) \cdot \text{chunkSize})$. Nobena druga nit ne piše v iste indekse. To je primer **prostorske dekompozicije** brez potrebe po zaklepanju.

Možno vprašanje na zagovoru

Kako zagotovite, da primerjate paralelizacijo in ne razlik med implementacijami?

Odgovor: Vsi trije načini kličejo **isti metodi** `ForceKernel.computeForces()` in `ForceKernel.integrate()` nad **isto podatkovno postavitevjo**, in tečejo v **isti zanki** `SimulationRunner.runCycles()`. Razlikujejo se samo v mejah območja, ki ga dobi vsak izvajalec, in v načinu sinhronizacije na koncu cikla. Če bi vsak način imel svojo kopijo fizike, bi izmerjena razmerja odražala razlike med temi kopijami in ne učinka paralelizacije.

Možno vprašanje na zagovoru

Zakaj so rezultati vseh treh načinov **popolnoma** enaki, ne le približno?

Odgovor: Sile na delec i se v vseh načinih seštevajo v **istem vrstnem redu** ($j = 0 \dots n$), saj vsak izvajalec za svoj delec vedno pregleda vse delce po naraščajočem indeksu. Ker se seštevanje v plavajoči vejici nikjer ne prerazporedi, ni razlik niti v zadnjem bitu. Razdelitev dela vpliva na to, **kdo** računa, ne na to, **v kakšnem vrstnem redu** seštevajo.

Možno vprašanje na zagovoru

Zakaj pohitritev ne doseže $10\times$ na 10-jedrnem procesorju?

Odgovor: Apple M4 ima **4 zmogljiva (P) in 6 varčnih (E) jeder**. Prve štiri niti dobijo P-jedra in prispevajo skoraj polno zmogljivost, naslednjih šest pa E-jedra, ki so bistveno počasnejša. Ker so kosi **enako veliki** in je na koncu vsakega cikla pregrada, vsi čakajo na najpočasnejšega. Zato učinkovitost pri 4 izvajalcih (51–73 %) močno presega učinkovitost pri 10 izvajalcih (22–29 %).